Mecanisme de Self Attention - Noé Breton

- Mecanisme de Self Attention
    - Exercice 1 : Implementation du mécanisme de self-attention
        - Créez des embeddings pour chaque mots
        - Calculez les vecteurs Query, Key, et Value
        - Calculez les scores d'attention
        - Appliquez la softmax
        - Obtenez la sortie pondérée
        - Affichez les résultats
            - Ponderation
        - Vecteurs Finaux
        - Implementation du self attention
    - Exercice 2 : Mise en œuvre complète d'une attention multi-têtes avec encodage de position
        - a. Encodage de position :
            - Calcul des embedings
            - Ajout des vecteurs de positions au embedings
        - b. Mécanisme de self-attention multi-têtes :
        - Visualisation :
        - Bonus (facultatif) :
    - Limitation et Conculsion
        - Comment faire pour ameliorer le resultat ?
        - Conclusion

Exercice 1 : Implementation du mécanisme de self-attention

Créez des embeddings pour chaque mots

In [21]:

```python
import numpy as np
import string

def get_embeddings(text_input,d_embeding):
    #remove punctuation for having words
    text= "".join(letter for letter in text_input if letter not in string.punctuation)
    input_embeding={word :np.random.rand(d_embeding) for word in text.split(" ")}
    return input_embeding


text="Le chat noir dort."

d_embeding=4
# remove punctuation
# convert to random embedings
embedings_dict=get_embeddings(text,d_embeding)
words=list(embedings_dict.keys())
embedings_dict
```

Out[21]:

```
{'Le': array([0.22970219, 0.7479706 , 0.93397472, 0.74543035]),
 'chat': array([0.76768341, 0.02747404, 0.22829739, 0.12251647]),
 'noir': array([0.17446186, 0.74871488, 0.46027069, 0.49731285]),
 'dort': array([0.56343729, 0.74456453, 0.12211113, 0.98929841])}
```

Calculez les vecteurs Query, Key, et Value

In [22]:

```python
# random value weight matrix for d_head =2
W_Q,W_K,W_V =[np.random.rand(4,2) for _ in range(3)]

#convert the emdebings to np array
embedings=np.array(list(embedings_dict.values()))

#calculate vector Query Key and Value for each embdeding
Q,K,V=[embedings @ weight for weight in [W_Q,W_K,W_V]]

Q,K,V
```

Out[22]:

```
(array([[2.11129057, 1.77625586],
        [0.80652385, 0.8271746 ],
        [1.45351475, 1.34916605],
        [1.69823171, 2.01988739]]),
 array([[0.77046571, 2.20409206],
        [0.15115707, 0.71982327],
        [0.56485945, 1.58565257],
        [0.58627826, 1.87377597]]),
 array([[1.07824654, 1.03506178],
        [0.65455296, 0.32854827],
        [0.61763145, 0.65761614],
        [0.64969853, 1.11851674]]))
```

Calculez les scores d'attention

In [23]:

```python
scores=Q @ K.T
scores

for i in range(4):
    for j in range(4):
        print(words[i],words[j],scores[i][j])
```

```
Le Le 5.5417084165924635
Le chat 1.5977268024865614
Le noir 4.009107098480169
Le dort 4.566109298258204
chat Le 2.4445679482995266
chat chat 0.7173313133030152
chat noir 1.7671841579245522
chat dort 2.022787294192136
noir Le 4.093569458040712
noir chat 1.1908701559120407
noir noir 2.960340164949784
noir dort 3.380199025106921
dort Le 5.760447054146966
dort chat 1.710661680419594
dort noir 4.162101863354266
dort dort 4.780452777286102
```

Appliquez la softmax

In [24]:

```python
softmax_results=np.exp(scores)/np.sum(np.exp(scores),axis=1,keepdims=True)
softmax_results
```

Out[24]:

```
array([[0.62022793, 0.01201439, 0.13395256, 0.23380511],
       [0.42705896, 0.0759204 , 0.2169221 , 0.28009853],
       [0.53565978, 0.02939427, 0.17247804, 0.2624679 ],
       [0.62697094, 0.01092567, 0.12679289, 0.2353105 ]])
```

Obtenez la sortie pondérée

In [25]:

```python
ponderated_sofmax=softmax_results @ V
ponderated_sofmax
```

Out[25]:

```
array([[0.91125883, 0.99552583],
       [0.82612629, 0.9229223 ],
       [0.87386629, 0.97109749],
       [0.91437284, 0.99912305]])
```

Affichez les résultats

Ponderation

In [26]:

```python
for i,word in enumerate(embedings_dict.keys()):
    print(word,softmax_results[i] )
```

```
Le [0.62022793 0.01201439 0.13395256 0.23380511]
chat [0.42705896 0.0759204  0.2169221  0.28009853]
noir [0.53565978 0.02939427 0.17247804 0.2624679 ]
dort [0.62697094 0.01092567 0.12679289 0.2353105 ]
```

Vecteurs Finaux

In [27]:

```python
for i,word in enumerate(embedings_dict.keys()):
    print(word,ponderated_sofmax[i] )
```

```
Le [0.91125883 0.99552583]
chat [0.82612629 0.9229223 ]
noir [0.87386629 0.97109749]
dort [0.91437284 0.99912305]
```

Implementation du self attention

In [28]:

```python
def self_attention_one_head(input_text,d_embeding,d_head):
    embedings_dict=get_embeddings(input_text,d_embeding)
    #weight matrix
    W_Q,W_K,W_V =[np.random.rand(d_embeding,d_head) for _ in range(3)]
    #convert the emdebings to np array
    embedings=np.array(list(embedings_dict.values()))
    #calculate vector Query Key and Value for each embdeding
    Q,K,V=[embedings @ weight for weight in [W_Q,W_K,W_V]]
    #calculate scores
    scores=Q @ K.T
    #applie softmax
    ponderate_results=np.exp(scores)/np.sum(np.exp(scores),axis=1,keepdims=True)
    final_vectors=ponderate_results@V
    # Les mots de l'entrée

    indexed_vectors = dict(zip(embedings_dict.keys(), final_vectors))
    return indexed_vectors
```

In [29]:

```python
self_attention_one_head("Le chat noir dort.",4,2)
```

Out[29]:

```
{'Le': array([1.40028801, 1.4866266 ]),
 'chat': array([1.34792648, 1.43095662]),
 'noir': array([1.36532797, 1.44862056]),
 'dort': array([1.34597628, 1.42852343])}
```

Exercice 2 : Mise en œuvre complète d'une attention multi-têtes avec encodage de position

a. Encodage de position :

In [30]:

```python
input_text="Le soleil brille doucement sur la rivière."
words="".join(letter for letter in input_text if letter not in string.punctuation).split()
n_head=2
d_head=3
d_embeding=6
words
```

Out[30]:

```
['Le', 'soleil', 'brille', 'doucement', 'sur', 'la', 'rivière']
```

In [31]:

```python
import numpy as np

def positional_encoding(max_position, d_model):
    # Créer une matrice vide
    encoding = np.zeros((max_position, d_model))
    for pos in range(max_position):
        for i in range(0, d_model, 2):  # Dimensions paires
            encoding[pos, i] = np.sin(pos / (10000 ** (i / d_model)))
            encoding[pos, i + 1] = np.cos(pos / (10000 ** (i / d_model)))
    return encoding

# Exemple : encoding des position de notre phrase.
pos = positional_encoding(len(words), d_embeding)
pos
```

Out[31]:

```
array([[ 0.        ,  1.        ,  0.        ,  1.        ,  0.        ,
         1.        ],
       [ 0.84147098,  0.54030231,  0.04639922,  0.99892298,  0.00215443,
         0.99999768],
       [ 0.90929743, -0.41614684,  0.0926985 ,  0.99569422,  0.00430886,
         0.99999072],
       [ 0.14112001, -0.9899925 ,  0.1387981 ,  0.9903207 ,  0.00646326,
         0.99997911],
       [-0.7568025 , -0.65364362,  0.18459872,  0.98281398,  0.00861763,
         0.99996287],
       [-0.95892427,  0.28366219,  0.23000171,  0.97319022,  0.01077197,
         0.99994198],
       [-0.2794155 ,  0.96017029,  0.27490927,  0.96147017,  0.01292625,
         0.99991645]])
```

Calcul des embedings

In [32]:

```python
embedings_dict=get_embeddings(input_text,d_embeding)
embedings_dict
```

Out[32]:

```
{'Le': array([0.88109913, 0.1653453 , 0.32040311, 0.72455212, 0.1387932 ,
        0.45601837]),
 'soleil': array([0.34907011, 0.30053935, 0.44891854, 0.17655506, 0.6848809 ,
        0.60650251]),
 'brille': array([0.49193009, 0.94939179, 0.41334387, 0.89658353, 0.73064361,
        0.18047423]),
 'doucement': array([0.85163362, 0.85804569, 0.78824315, 0.22749316, 0.5838839 ,
        0.12592592]),
 'sur': array([0.05264462, 0.65211038, 0.92168775, 0.49991726, 0.37423097,
        0.61317421]),
 'la': array([0.65032864, 0.09201503, 0.11713496, 0.34743637, 0.72765083,
        0.83537362]),
 'rivière': array([0.77462308, 0.58430745, 0.03189382, 0.09074873, 0.32279182,
        0.87518444])}
```

Ajout des vecteurs de positions au embedings

In [33]:

```python
#on recupere un vecteur numpy pour optimiser les calculs :
embedings=np.array(list(embedings_dict.values()))
#on ajoute la position
embedings=embedings+pos
embedings
```

Out[33]:

```
array([[ 8.81099127e-01,  1.16534530e+00,  3.20403112e-01,
         1.72455212e+00,  1.38793202e-01,  1.45601837e+00],
       [ 1.19054109e+00,  8.40841655e-01,  4.95317765e-01,
         1.17547803e+00,  6.87035331e-01,  1.60650019e+00],
       [ 1.40122752e+00,  5.33244954e-01,  5.06042373e-01,
         1.89227775e+00,  7.34952469e-01,  1.18046494e+00],
       [ 9.92753630e-01, -1.31946803e-01,  9.27041249e-01,
         1.21781386e+00,  5.90347163e-01,  1.12590503e+00],
       [-7.04157880e-01, -1.53323988e-03,  1.10628647e+00,
         1.48273124e+00,  3.82848604e-01,  1.61313708e+00],
       [-3.08595640e-01,  3.75677217e-01,  3.47136668e-01,
         1.32062659e+00,  7.38422795e-01,  1.83531560e+00],
       [ 4.95207586e-01,  1.54447773e+00,  3.06803087e-01,
         1.05221890e+00,  3.35718064e-01,  1.87510089e+00]])
```

b. Mécanisme de self-attention multi-têtes :

In [34]:

```python
#initialiser les matrices de poids pour les deux têtes
W_Q=[np.random.rand(d_embeding,d_head) for _ in range(n_head)]
W_K=[np.random.rand(d_embeding,d_head) for _ in range(n_head)]
W_V =[np.random.rand(d_embeding,d_head) for _ in range(n_head)]

Q = [embedings @ wq for wq in W_Q]  # Liste des Q pour chaque tête
K = [embedings @ wk for wk in W_K]  # Liste des K pour chaque tête
V = [embedings @ wv for wv in W_V]  # Liste des V pour chaque tête
Q,K,V
#calculate vector Query Key and Value for each embdeding
# Q[0] is Q for head 1 and Q[1] is Q for head 2
```

Out[34]:

```
([array([[1.76396533, 4.08612139, 2.22532872],
         [1.98257096, 4.14500347, 2.63262969],
         [2.08697904, 4.52084885, 2.43494383],
         [1.63144177, 3.42290536, 2.15901113],
         [1.26352291, 2.46325974, 2.36596253],
         [1.54204112, 2.75958281, 2.5647581 ],
         [1.70560606, 3.66196966, 2.54870237]]),
  array([[2.62053184, 2.28576133, 2.27199826],
         [2.33275408, 2.11543289, 2.59623438],
         [2.32634468, 2.09002096, 2.44561233],
         [1.43988885, 1.11066935, 2.15779093],
         [1.43506013, 0.52226888, 2.2701701 ],
         [1.75124529, 0.95560228, 2.4093546 ],
         [2.66245781, 2.34922042, 2.49641655]])],
 [array([[2.60550427, 3.8479151 , 2.46832569],
         [2.45063984, 3.89319624, 2.3744497 ],
         [3.09920573, 4.33573482, 2.8176985 ],
         [2.17679684, 3.3853927 , 2.27491077],
         [1.75455115, 2.76890285, 2.09148341],
         [1.96281248, 2.70646293, 1.92629384],
         [2.01497515, 3.4530921 , 2.02334361]]),
  array([[2.59051064, 1.25831487, 3.46688378],
         [3.12763201, 1.46320161, 3.5655649 ],
         [2.87387114, 1.66611402, 3.87024383],
         [2.33232591, 1.75551535, 2.86400353],
         [1.55895697, 1.46972963, 2.01461263],
         [2.09006216, 0.87547514, 2.30389695],
         [2.85602481, 1.02119655, 3.1827289 ]])],
 [array([[4.03630314, 2.22110356, 3.26439624],
         [4.09789679, 2.34015841, 3.2720428 ],
         [4.18375558, 2.54111989, 3.80185321],
         [3.18696276, 1.87565832, 2.58534987],
         [3.12863314, 0.66593118, 1.31234278],
         [3.34472381, 0.88549356, 1.72289015],
         [4.0817763 , 1.90745962, 2.74364989]]),
  array([[2.65349533, 2.99076422, 3.47628468],
         [3.15619895, 2.90142807, 3.67293531],
         [3.37086334, 3.15533591, 3.2512129 ],
         [2.88315911, 2.27177071, 2.51539042],
         [2.23215094, 1.88488927, 2.7229236 ],
         [2.36058395, 1.93201218, 2.97764397],
         [2.51590243, 2.80568123, 4.05277819]])])
```

In [35]:

```python
def self_attention(Q,V,K):
    scores = Q @ K.T
    softmax = np.exp(scores) / np.sum(np.exp(scores), axis=1, keepdims=True)
    return softmax @ V,scores

heads=[]
scores=[]
for q, k, v in zip(Q, K, V):
    head, score =self_attention(q, k, v)
    heads.append(head)
    scores.append(score)

multi_head_output = np.concatenate(heads, axis=1)

print("Résultat de l'attention multi-têtes:\n", multi_head_output)
```

```
Résultat de l'attention multi-têtes:
 [[3.0033314  4.26136746 2.75132721 2.93853753 1.4792518  3.63700325]
 [3.02309197 4.27688659 2.76503354 2.93882442 1.43643255 3.5905053 ]
 [3.02282458 4.27691875 2.76486924 2.93678711 1.44435583 3.59867869]
 [2.98322125 4.244979   2.73730843 2.90492294 1.36526106 3.5009469 ]
 [2.96423858 4.22867356 2.72404945 2.89839703 1.34103796 3.46353955]
 [2.9873133  4.24759048 2.74009421 2.9194817  1.37053902 3.50833307]
 [3.00856645 4.26514627 2.75493018 2.94228484 1.47133525 3.62869615]]
```

Visualisation :

Affichez les scores d'attention pour chaque mot (matrice nwords× nwords). o Interprétez les relations entre les mots basées sur les scores.

In [36]:

```python
for n_head, score_matrix in enumerate(scores):
    print(f"Attention score head n° {n_head + 1} :")
    score_matrix = np.array(score_matrix)
    print("      ", "  ".join([f"{word}" for word in words]))
    for i in range(len(words)):
        row = [f"{score_matrix[i][j]:.2f}" for j in range(len(words))]
        print(f"{words[i][:3]} -> ", "  ".join(row))
    print()
```

```
Attention score head n° 1 :
       Le  soleil  brille  doucement  sur  la  rivière
Le ->  23.46  24.07  26.22  19.04  11.16  13.35  21.10
sol ->  25.80  26.44  28.84  20.90  12.42  14.84  23.22
bri ->  26.41  27.10  29.48  21.43  12.74  15.18  23.82
dou ->  21.24  21.76  23.73  17.20  10.22  12.21  19.11
sur ->  18.29  18.68  20.54  14.76  8.70  10.48  16.35
la ->  20.73  21.17  23.21  16.72  10.03  12.02  18.59
riv ->  23.34  23.90  26.13  18.89  11.12  13.34  20.94

Attention score head n° 2 :
       Le  soleil  brille  doucement  sur  la  rivière
Le ->  21.69  23.25  23.43  18.46  16.34  17.37  22.21
sol ->  21.54  23.04  22.98  18.06  16.26  17.32  22.33
bri ->  20.93  22.39  22.39  17.61  15.79  16.81  21.63
dou ->  14.64  15.69  15.37  12.10  11.18  11.97  15.48
sur ->  13.26  14.38  13.87  11.03  10.37  11.16  14.28
la ->  15.88  17.15  16.75  13.28  12.27  13.15  16.85
riv ->  22.77  24.39  24.50  19.29  17.17  18.26  23.41
```

In [37]:

```python
#affiche le top 3 des relation
def display_top_rel(scores,words,n_top):
    for i, score in enumerate(scores):
        print(f"\nTête {i + 1}:")
        top_relations={}
        for word_idx, word in enumerate(words):
            sorted_indices = np.argsort(score[word_idx])[::-1]
            top_indices = [idx for idx in sorted_indices if idx != word_idx][:n_top]
            top_relations[word] = [(words[idx], score[word_idx, idx]) for idx in top_indices]
            top_relations_str = ", ".join([f"'{w}' ({s:.2f})" for w, s in top_relations[word]])
            print(f"  {word}: {top_relations_str}")
    return top_relations
print("\nTop 3 relations pour chaque mot (excluant les relations entre un mot et lui-même) :")
display_top_rel(scores,words,3);
```

```
Top 3 relations pour chaque mot (excluant les relations entre un mot et lui-même) :

Tête 1:
  Le: 'brille' (26.22), 'soleil' (24.07), 'rivière' (21.10)
  soleil: 'brille' (28.84), 'Le' (25.80), 'rivière' (23.22)
  brille: 'soleil' (27.10), 'Le' (26.41), 'rivière' (23.82)
  doucement: 'brille' (23.73), 'soleil' (21.76), 'Le' (21.24)
  sur: 'brille' (20.54), 'soleil' (18.68), 'Le' (18.29)
  la: 'brille' (23.21), 'soleil' (21.17), 'Le' (20.73)
  rivière: 'brille' (26.13), 'soleil' (23.90), 'Le' (23.34)

Tête 2:
  Le: 'brille' (23.43), 'soleil' (23.25), 'rivière' (22.21)
  soleil: 'brille' (22.98), 'rivière' (22.33), 'Le' (21.54)
  brille: 'soleil' (22.39), 'rivière' (21.63), 'Le' (20.93)
  doucement: 'soleil' (15.69), 'rivière' (15.48), 'brille' (15.37)
  sur: 'soleil' (14.38), 'rivière' (14.28), 'brille' (13.87)
  la: 'soleil' (17.15), 'rivière' (16.85), 'brille' (16.75)
  rivière: 'brille' (24.50), 'soleil' (24.39), 'Le' (22.77)
```

On remarque que sur les deux têtes, les mots "Le" et "soleil" restent dans le top 3 des scores de relation, malgré les variations aléatoires. Cela pourrait indiquer que ces mots ont une relation particulièrement significative dans le contexte. De plus, le mot suivant, "brille", apparaît également très fréquemment dans les relations les plus élevées, ce qui pourrait signaler une forte connexion entre ces mots.

Les deux têtes n'ont pas exactement les mêmes résultats, on observe que les scores varient, mais les similarités entre les mots restent relativement cohérentes. Les relations observées ne sont donc pas complètement différentes entre les têtes, ce qui suggère que le modèle apprend une représentation stable des relations contextuelles entre ces mots, indépendamment des variations aléatoires.

Bonus (facultatif) :

On definis la fonction multiheaded_attention(text_input, n_head, d_head,)

In [38]:

```python
def multi_head_attention(input_text,d_embeding,d_head,n_head):
    words="".join(letter for letter in input_text if letter not in string.punctuation).split()
    embedings_dict=get_embeddings(input_text,d_embeding)
    embedings=np.array(list(embedings_dict.values()))
    #on ajoute la position
    embedings_with_pos=embedings+pos
    #on calcule n_head matrice de poid d_embeding x d_head
    W_Q=[np.random.rand(d_embeding,d_head) for _ in range(n_head)]
    W_K=[np.random.rand(d_embeding,d_head) for _ in range(n_head)]
    W_V =[np.random.rand(d_embeding,d_head) for _ in range(n_head)]
    Q = [embedings_with_pos @ wq for wq in W_Q]  # Liste des Q pour chaque tête
    K = [embedings_with_pos @ wk for wk in W_K]  # Liste des K pour chaque tête
    V = [embedings_with_pos @ wv for wv in W_V]  # Liste des V pour chaque tête
    heads=[]
    scores=[]
    for q, k, v in zip(Q, K, V):
        head, score =self_attention(q, k, v)
        heads.append(head)
        scores.append(score)
    multi_head_output = np.concatenate(heads, axis=1)
    return words,multi_head_output,scores

words, ponderate_results, scores =multi_head_attention("La lune brille doucement sur la rivière.",d_embeding=6,d_head=3,n_head=2)
```

On affiche le top 3 des relations les plus fortes pour chaque mots de l'entrée

In [39]:

```python
display_top_rel(scores,words,3);
```

```
Tête 1:
  La: 'lune' (26.12), 'brille' (21.25), 'rivière' (19.80)
  lune: 'brille' (26.37), 'La' (24.89), 'rivière' (24.15)
  brille: 'lune' (25.59), 'La' (19.85), 'rivière' (19.24)
  doucement: 'lune' (14.99), 'brille' (12.30), 'La' (11.46)
  sur: 'lune' (14.41), 'brille' (11.71), 'La' (11.22)
  la: 'lune' (18.40), 'brille' (14.94), 'La' (14.30)
  rivière: 'lune' (24.91), 'brille' (20.31), 'La' (19.20)

Tête 2:
  La: 'lune' (29.26), 'brille' (23.91), 'rivière' (22.86)
  lune: 'brille' (29.25), 'rivière' (28.11), 'La' (27.08)
  brille: 'lune' (28.40), 'rivière' (22.41), 'La' (21.56)
  doucement: 'lune' (20.09), 'brille' (16.34), 'rivière' (15.84)
  sur: 'lune' (18.75), 'brille' (15.27), 'rivière' (14.76)
  la: 'lune' (23.46), 'brille' (19.15), 'rivière' (18.38)
  rivière: 'lune' (30.67), 'brille' (25.07), 'La' (23.08)
```

En remplaçant "Le soleil" par "La lune", on observe de légères différences dans les scores de relation entre chaque mot de la phrase. Par exemple, la relation entre "La" et "lune" est généralement moins forte que celle entre "Le" et "soleil". Mon interprétation est qu'il y a deux occurrences du mot "la" dans la phrase, ce qui signifie que "la" n'est pas uniquement relié à "lune", contrairement à "le" qui est directement associé à "soleil". Il est également possible que la majorité des mots féminins de la phrase, comme "la", "lune" et "rivière", uniformisent les scores dans la phrase modifiée.

Limitation et Conculsion

Comment faire pour ameliorer le resultat ?

Faire plus de prétraitement sur le texte d'entrée permettrait de réduire le bruit et de mieux contrôler l'importance de certains mots. Par exemple, des techniques comme la lemmatisation ou la suppression des mots inutiles pourraient aider à mieux comprendre les relations entre les mots. En plus, au lieu d'initialiser les poids et les embeddings de manière aléatoire, utiliser des embeddings pré-existants pour chaque mot ou token améliorerait la cohérence des résultats, car chaque mot aurait une représentation plus précise.

Augmenter la dimension des embeddings et le nombre de têtes d'attention permettrait aussi de mieux capter les relations entre les mots, bien que cela demanderait plus de puissance de calcul.

Conclusion

Cet exercice a permis de mieux comprendre comment les LLM peuvent comprendre les relations sémantiques des mots en les représentant sous forme de vecteurs dans un hyperespace et en appliquant une série d'opérations linéaires. En implémentant "from scratch" le mécanisme de self-attention et d'attention multi-têtes avec encodage des positions, j'ai pu saisir les enjeux de ces méthodes. Cela me permettra de les utiliser de manière plus pertinente dans la suite de mon apprentissage.